Experimenting with Vortices

Feel free to modify the `vortices` list to add more vortices, change their positions, or alter their charges:

python Copy

```python
vortices = [
    (-30, 0, 1),  # Vortex at (-30, 0) with charge +1
    (0, 30, -1),  # Vortex at (0, 30) with charge -1
    (30, 0, 1),   # Vortex at (30, 0) with charge +1
    (0, -30, -1)  # Vortex at (0, -30) with charge -1
]
```

Notes

- **Topological Charge (q)**: The charge determines the direction of phase winding. Positive charges wind counterclockwise, negative charges wind clockwise.
- **Stability and Parameters**: Adjust `dt`, `alpha`, and `beta` as needed for numerical stability and to explore different dynamical regimes.

Visual Interpretation

- **Magnitude Plot ($|A|$)**: Shows the locations of vortices as dark spots (where $|A|$ is minimal).
- **Phase Plot**: You can also visualize the phase of A to see the winding around vortices. To do this, modify the plotting section:

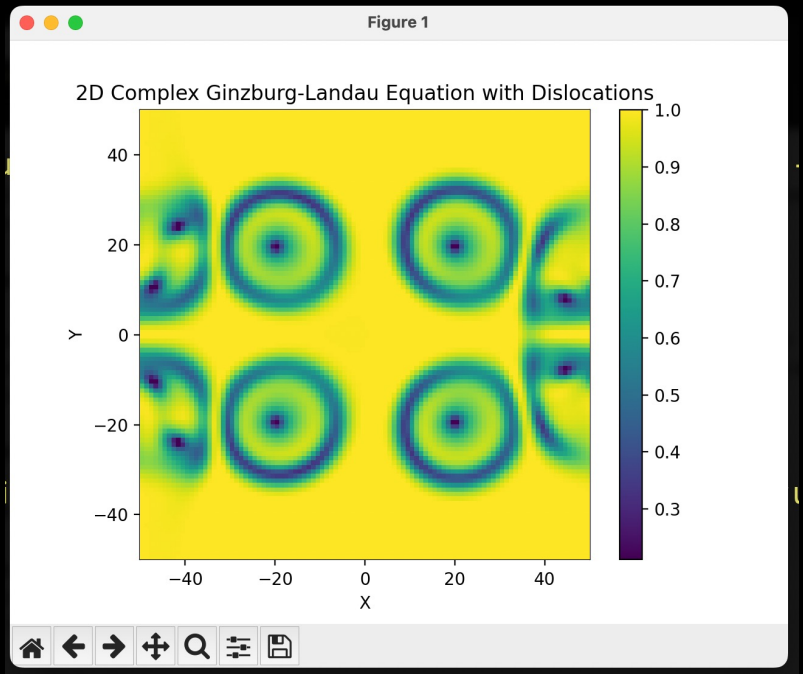
python Copy

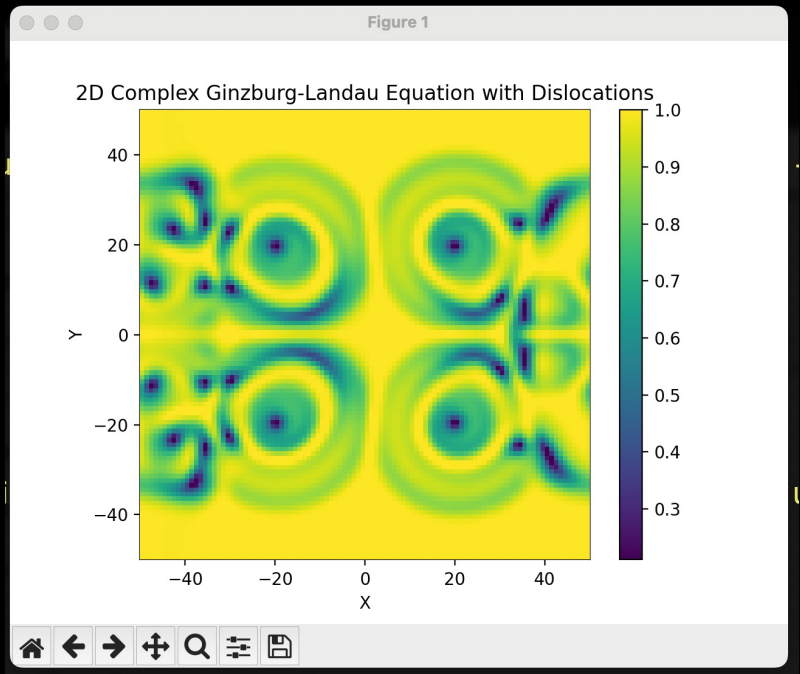
```python
im = ax.imshow(np.angle(A), animated=True, cmap='hsv', extent=[-nx/2, nx/2, -ny/2, ny/2])
```

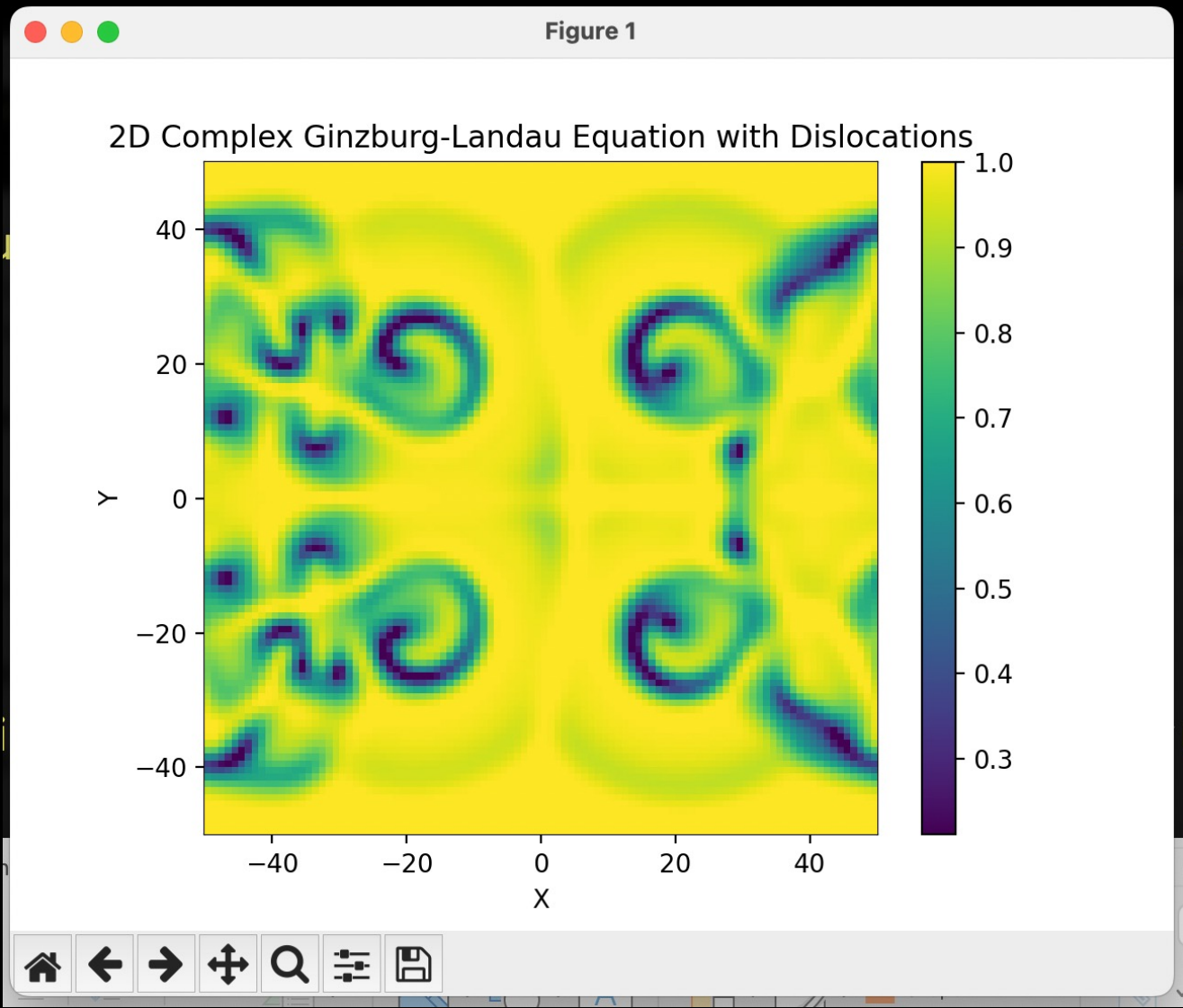
Conclusion

By seeding the initial condition with vortices, we've introduced dislocations into the superconducting lattice simulation. This allows you to study the dynamics of vortices, their interactions, and their impact on the system's evolution.

Let me know if you have any questions or need further customization!


Figure 1
2D Complex Ginzburg-Landau Equation with Dislocations
40
20
0
−20
−40
Y
−40
−20
0
20
40
X
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3


Figure 1
2D Complex Ginzburg-Landau Equation with Dislocations
40
20
0
−20
−40
Y
−40
−20
0
20
40
X
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3


Figure 1
2D Complex Ginzburg-Landau Equation with Dislocations
40
20
0
−20
−40
Y
−40
−20
0
20
40
X
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3

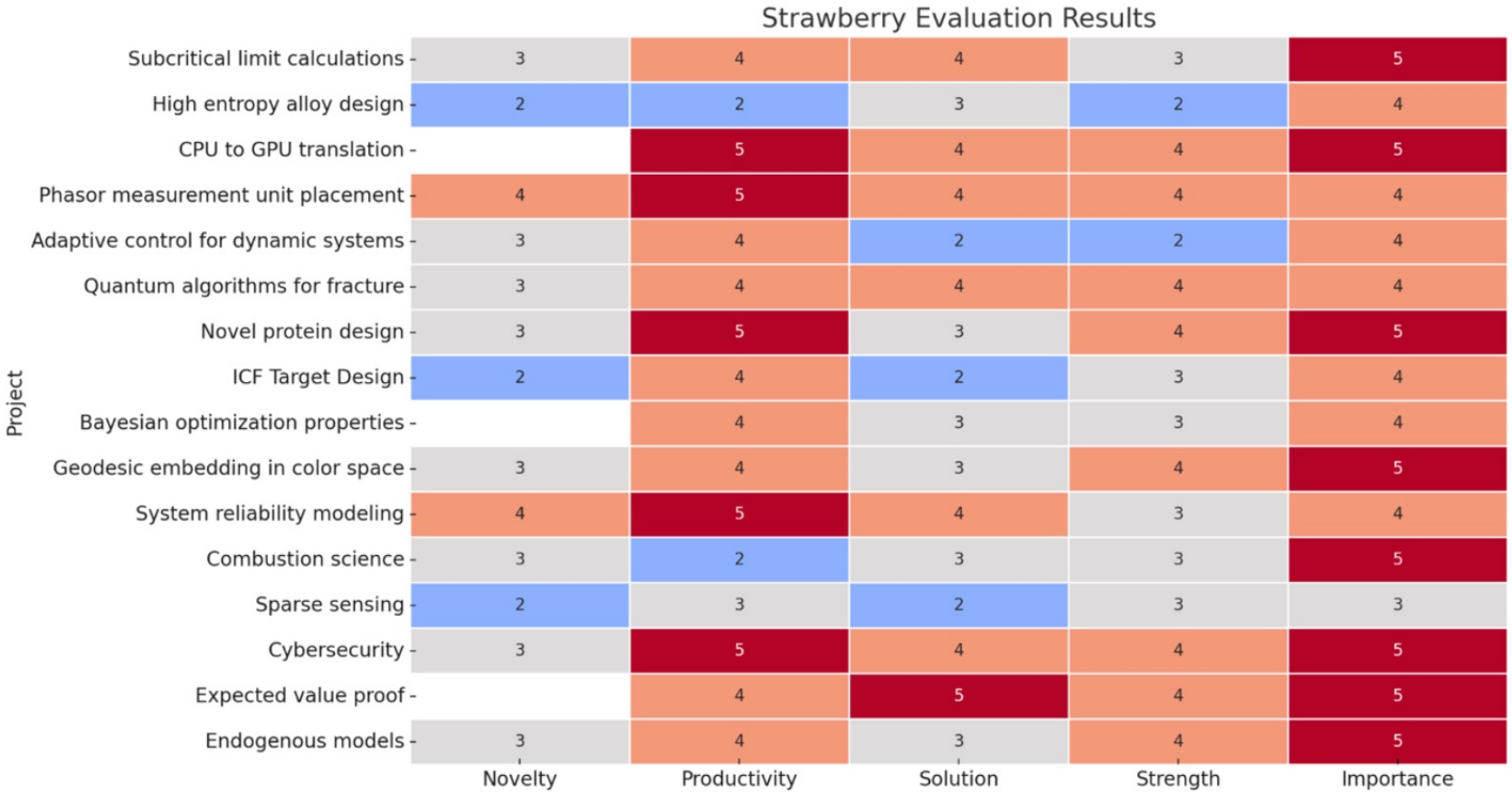
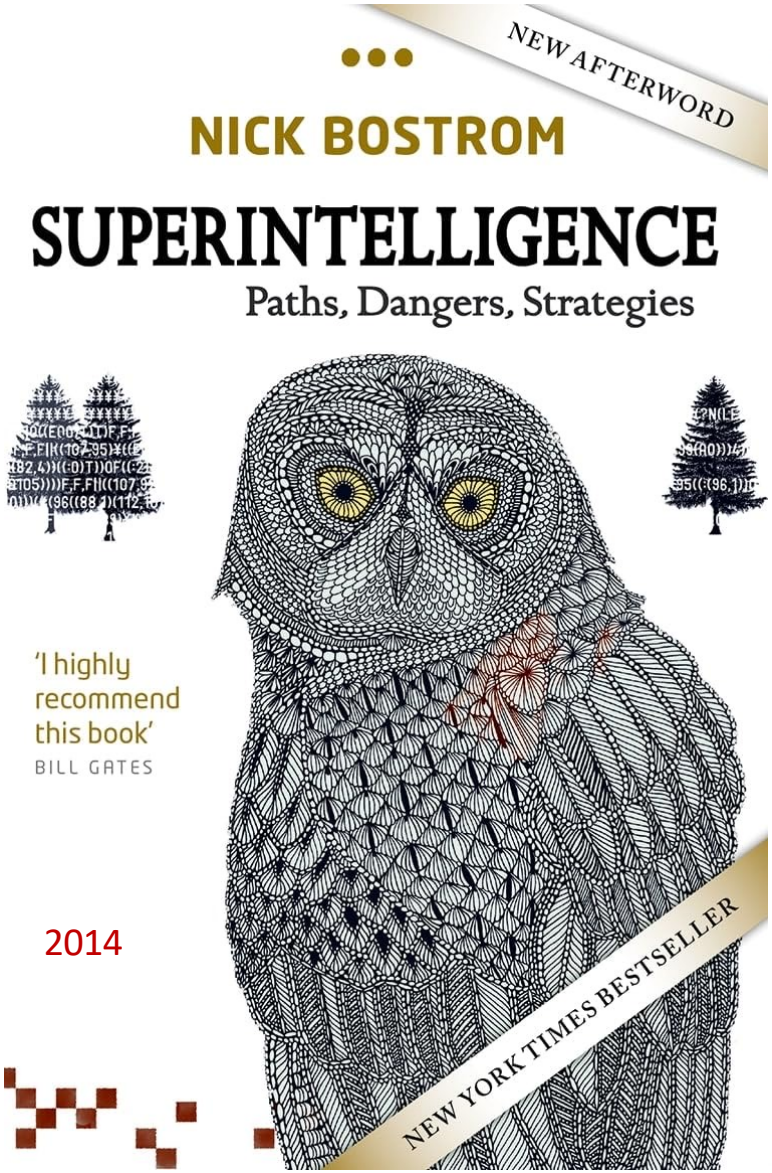


Figure 1 Participant scores for their projects. The scoring scales are given in appendix I. Missing scores in the 'Novelty' column correspond to cases where researchers were trying to reproduce complex, but already known, answers and did not feel they could assign a value.


Three routes to Superintelllience?



This response has highlighted statements because it was double-checked. How it works

Absolutely! Here's a summary of the types of superintelligence Nick Bostrom discusses in his book *Superintelligence: Paths, Dangers, Strategies*:

Types of Superintelligence

Bostrom doesn't outline strict categories, but focuses on a few core ways in which superintelligence could emerge:

- **Speed Superintelligence:** A system whose intellectual speed vastly outpaces any human mind. It could solve problems, perform research, and make decisions on timescales we couldn't comprehend. Even an otherwise 'human-level' intellect operating at extreme speed would have world-altering implications.
- **Collective Superintelligence:** A network of individuals (human or otherwise) whose collaborative thinking and problem-solving abilities outstrip those of any single member. Bostrom theorizes that large-scale networks with efficient coordination could become tremendously capable, akin to an organization operating a vast, super-efficient brain.
- **Quality Superintelligence:** An intellect that significantly surpasses humans in every single intellectual domain – creativity, strategic thinking, social skills, scientific reasoning, etc.

Gemini Advanced ULTRA

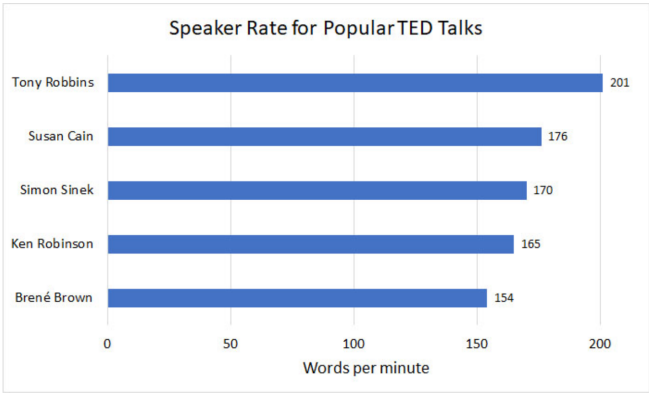
Speed SuperIntelligence

Here are some helpful rules of thumb for understanding tokens in terms of lengths: 1 token ~ 4 chars in English. 1 token ~ ¾ words. 100 tokens ~ 75 words.

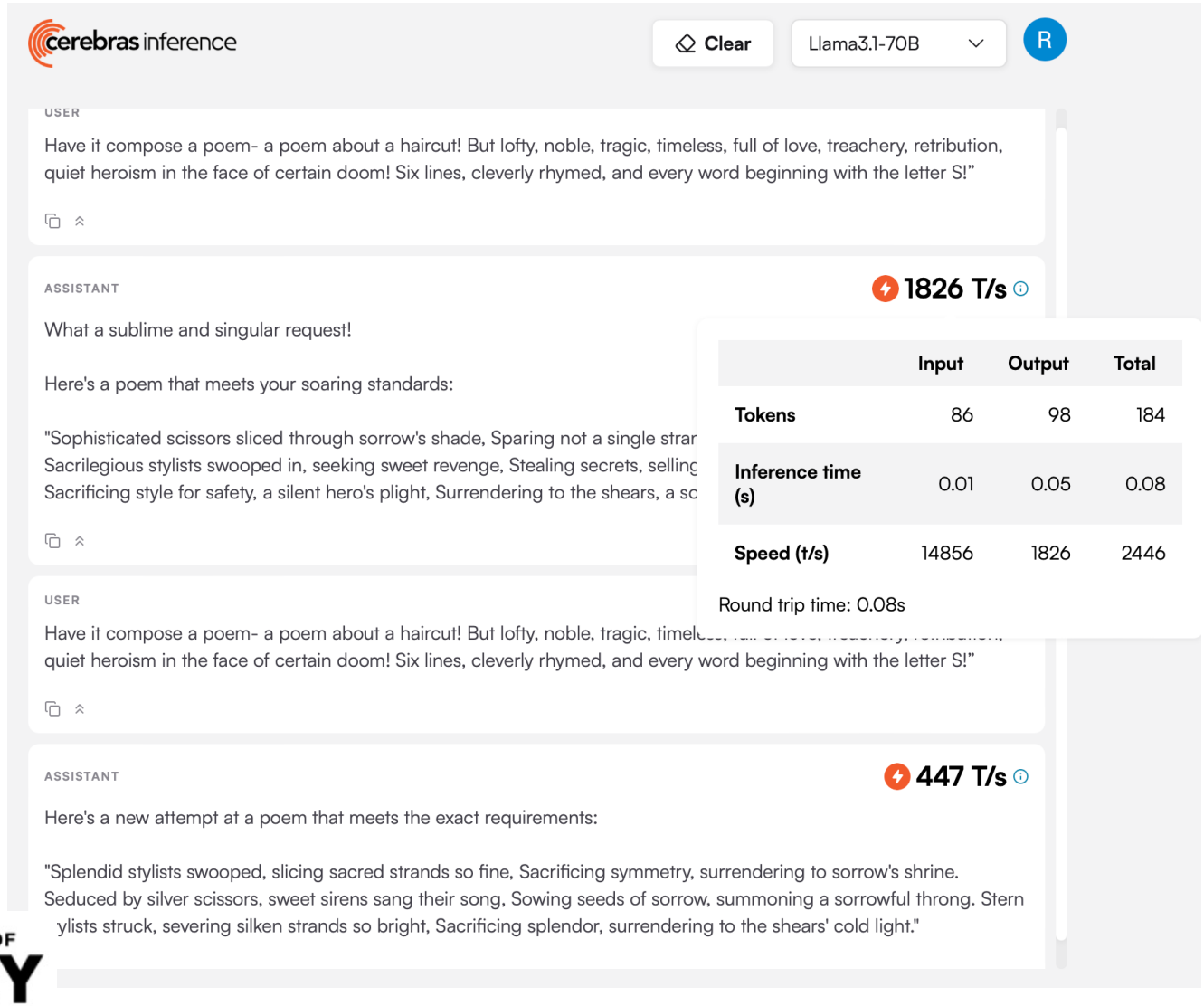
- AI that is a thousand times faster than a human?
- Humans speak about ~180 words per minute ~240 tokens/min (~4 t/s)
- Humans are estimated to ”think” about 2x-4x faster up to ~800 wpm (~20 t/s)
  - 6000+ thoughts per day ~ 1 million tokens/day ~ 160 tokens per thought?

Model	Input	Output
gpt-4-0125-preview	$0.01 / 1K tokens	$0.03 / 1K tokens
gpt-4-1106-preview	$0.01 / 1K tokens	$0.03 / 1K tokens
gpt-4-1106-vision-preview	$0.01 / 1K tokens	$0.03 / 1K tokens

- 1000x human ⇢ 20,000 t/s ⇢ 1 billion tokens/day
  - 6,000,000 thoughts per day!
- GPT4 costs ⇢ $0.03/1k sampled tokens
  - 1 million tokens/day ⇢ $30 per day
  - 1 billion tokens/day ⇢ $30,000 per day



U.S. DEPARTMENT OF ENERGY



U.S. DEPARTMENT OF ENERGY

Trend: Integrating networks of agents/resources

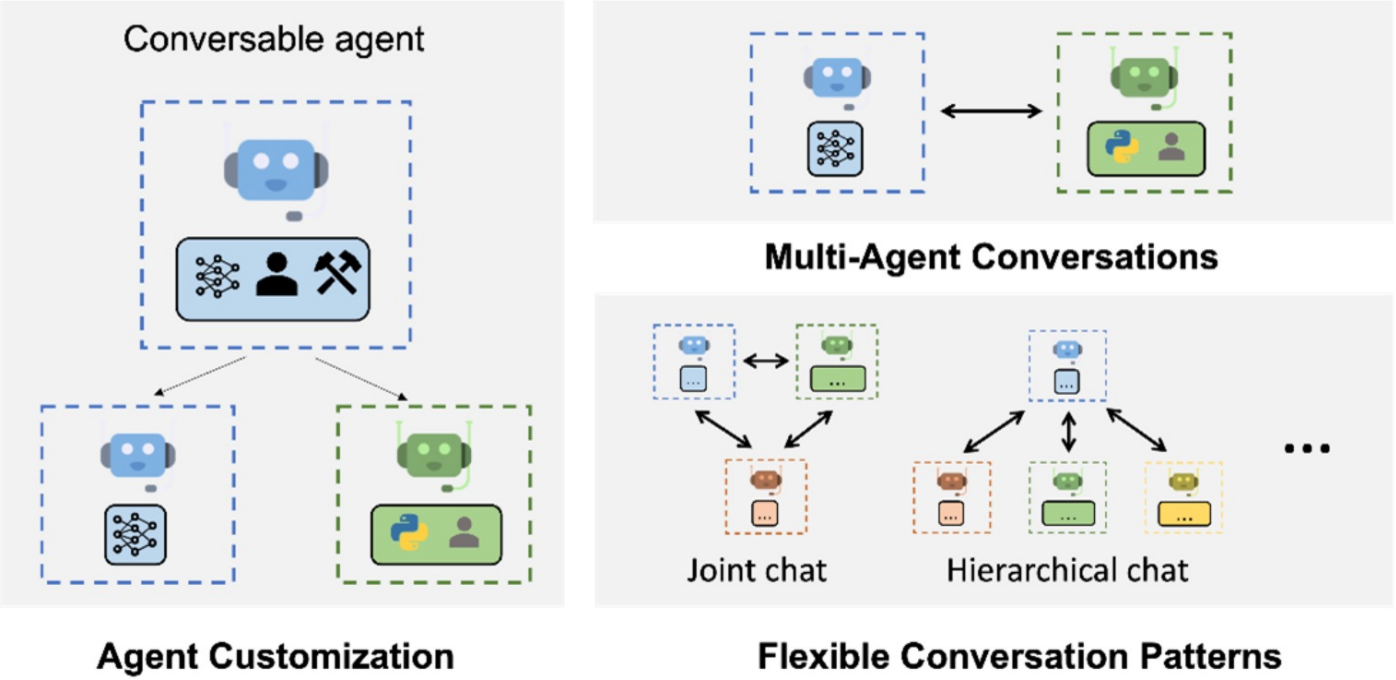


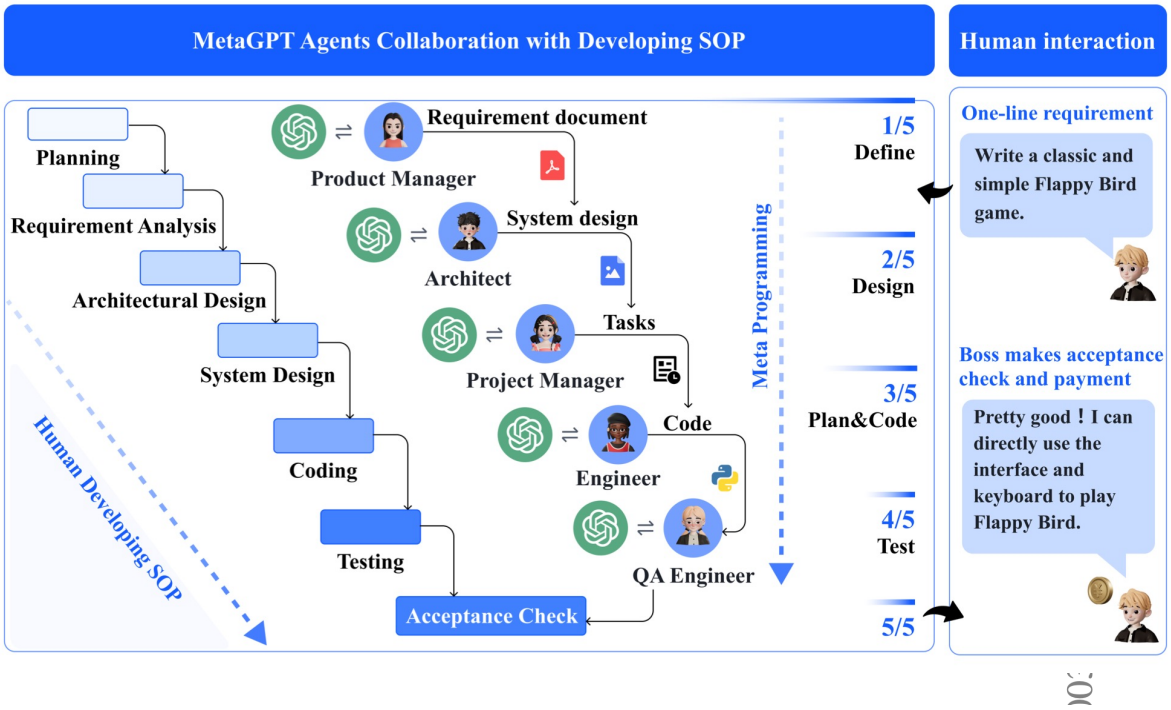
Figure 1. AutoGen enables complex LLM-based workflows using multi-agent conversations. (Left) AutoGen agents are customizable and can be based on LLMs, tools, humans, and even a combination of them. (Top-right) Agents can converse to solve tasks. (Bottom-right) The framework supports many additional complex conversation patterns.

https://techcommunity.microsoft.com/t5/educator-developer-blog/autogen-microsoft-s-open-source-tool-for-streamlining/ba-p/4040417

Networks of communicating AI agents that cooperatively solve problems

https://github.com/geekan/MetaGPT

Collective SuperIntelligence



10

https://arxiv.org/pdf/2308.00352.pdf

METAGPT: META PROGRAMMING FOR A MULTI-AGENT COLLABORATIVE FRAMEWORK

Sirui Hong[1,*], **Mingchen Zhuge**[2,*], **Jonathan Chen**[1], **Xiawu Zheng**[3], **Yuheng Cheng**[4], **Ceyao Zhang**[4], **Jinlin Wang**[1], **Zili Wang**, **Steven Ka Shing Yau**[5], **Zijuan Lin**[4], **Liyang Zhou**[6], **Chenyu Ran**[1], **Lingfeng Xiao**[1,7], **Chenglin Wu**[1,†], **Jürgen Schmidhuber**[2,8]

[1]DeepWisdom, [2]AI Initiative, King Abdullah University of Science and Technology, [3]Xiamen University, [4]The Chinese University of Hong Kong, Shenzhen, [5]Nanjing University, [6]University of Pennsylvania, [7]University of California, Berkeley, [8]The Swiss AI Lab IDSIA/USI/SUPSI

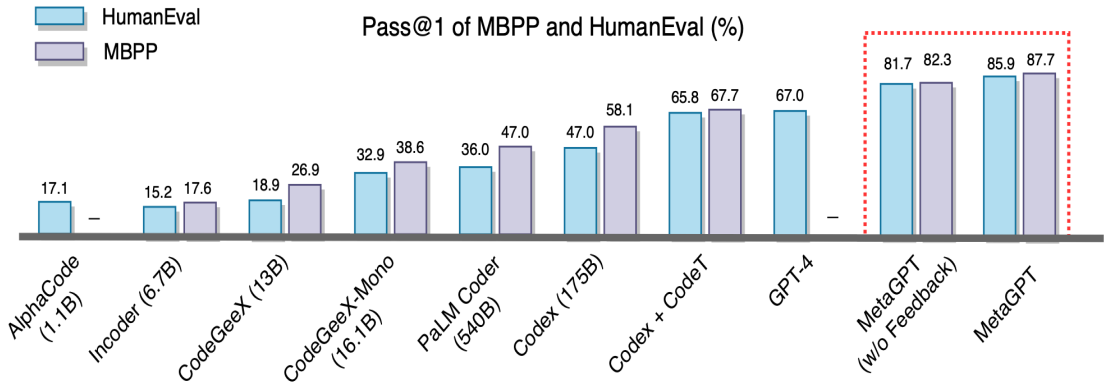
ABSTRACT

Remarkable progress has been made on automated problem solving through societies of agents based on large language models (LLMs). Existing LLM-based multi-agent systems can already solve simple dialogue tasks. Solutions to more complex tasks, however, are complicated through logic inconsistencies due to cascading hallucinations caused by naively chaining LLMs. Here we introduce MetaGPT, an innovative meta-programming framework incorporating efficient human workflows into LLM-based multi-agent collaborations. MetaGPT encodes Standardized Operating Procedures (SOPs) into prompt sequences for more streamlined workflows, thus allowing agents with human-like domain expertise to verify intermediate results and reduce errors. MetaGPT utilizes an assembly line paradigm to assign diverse roles to various agents, efficiently breaking down complex tasks into subtasks involving many agents working together. On collaborative software engineering benchmarks, MetaGPT generates more coherent solutions than previous chat-based multi-agent systems. Our project can be found at https://github.com/geekan/MetaGPT

“a personal AI team, full of virtual experts in different areas”

Collective SuperIntelligence

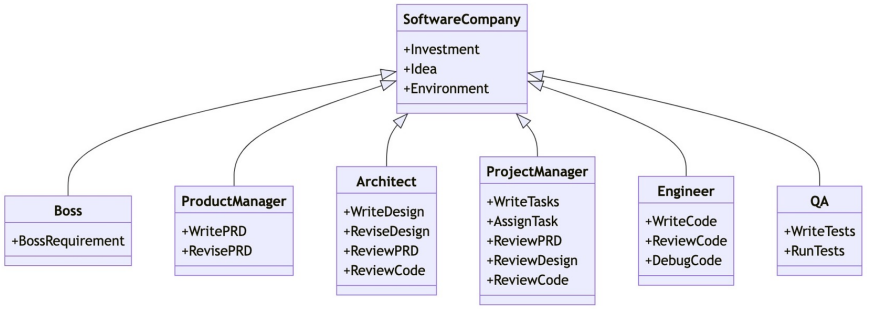
Create an entire company worth of AI agents with one prompt!



When MetaGPT collaborates with GPT-4, it significantly improves the Pass @k in the HumanEval benchmark compared to GPT-4. It achieves 85.9% and 87.7% in these two public benchmarks.

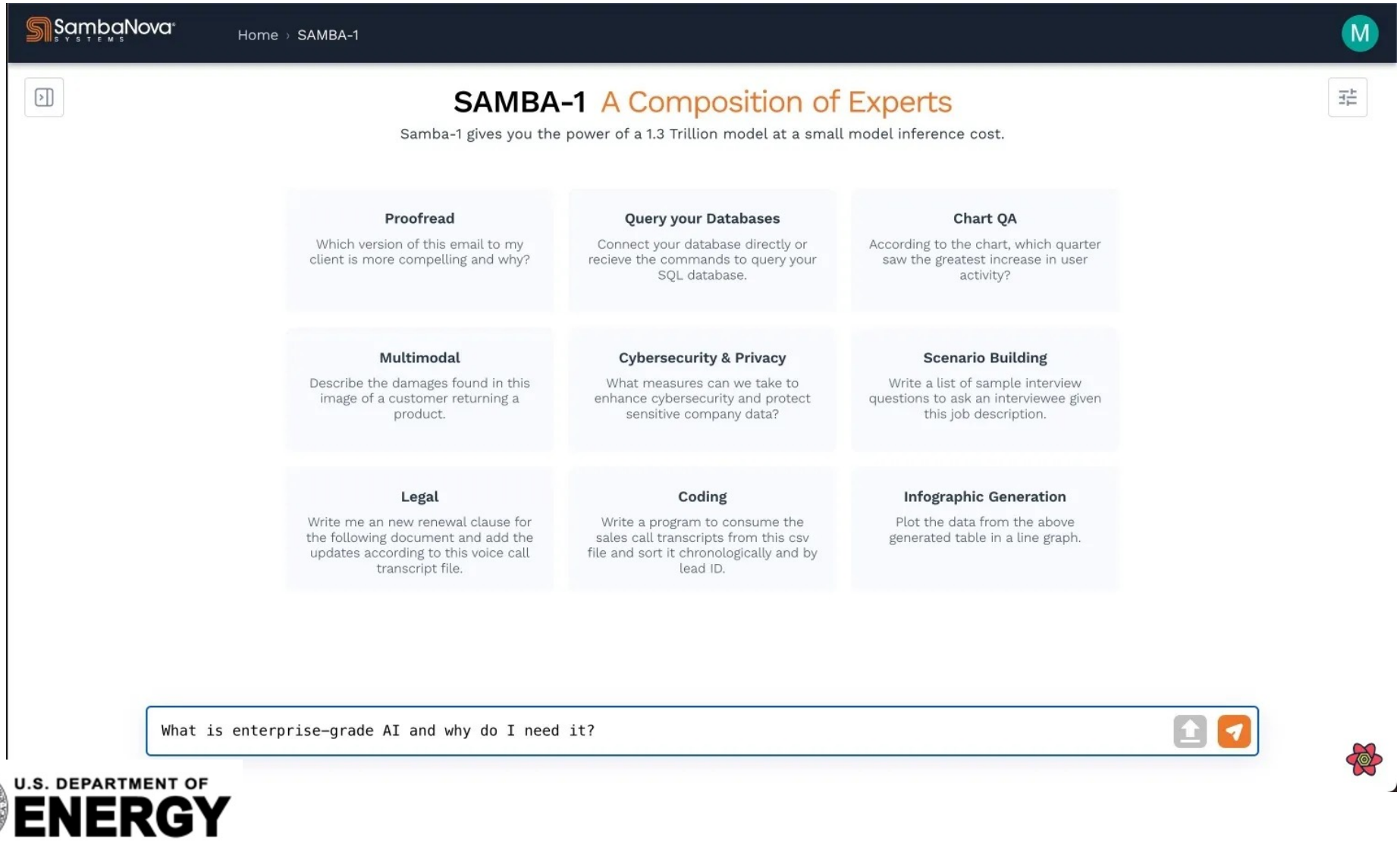
Software Company as Multi-Agent System

1. MetaGPT takes a **one line requirement** as input and outputs **user stories / competitive analysis / requirements / data structures / APIs / documents, etc.**
2. Internally, MetaGPT includes **product managers / architects / project managers / engineers.** It provides the entire process of a **software company along with carefully orchestrated SOPs.**
   i. `Code = SOP(Team)` is the core philosophy. We materialize SOP and apply it to teams composed of LLMs.



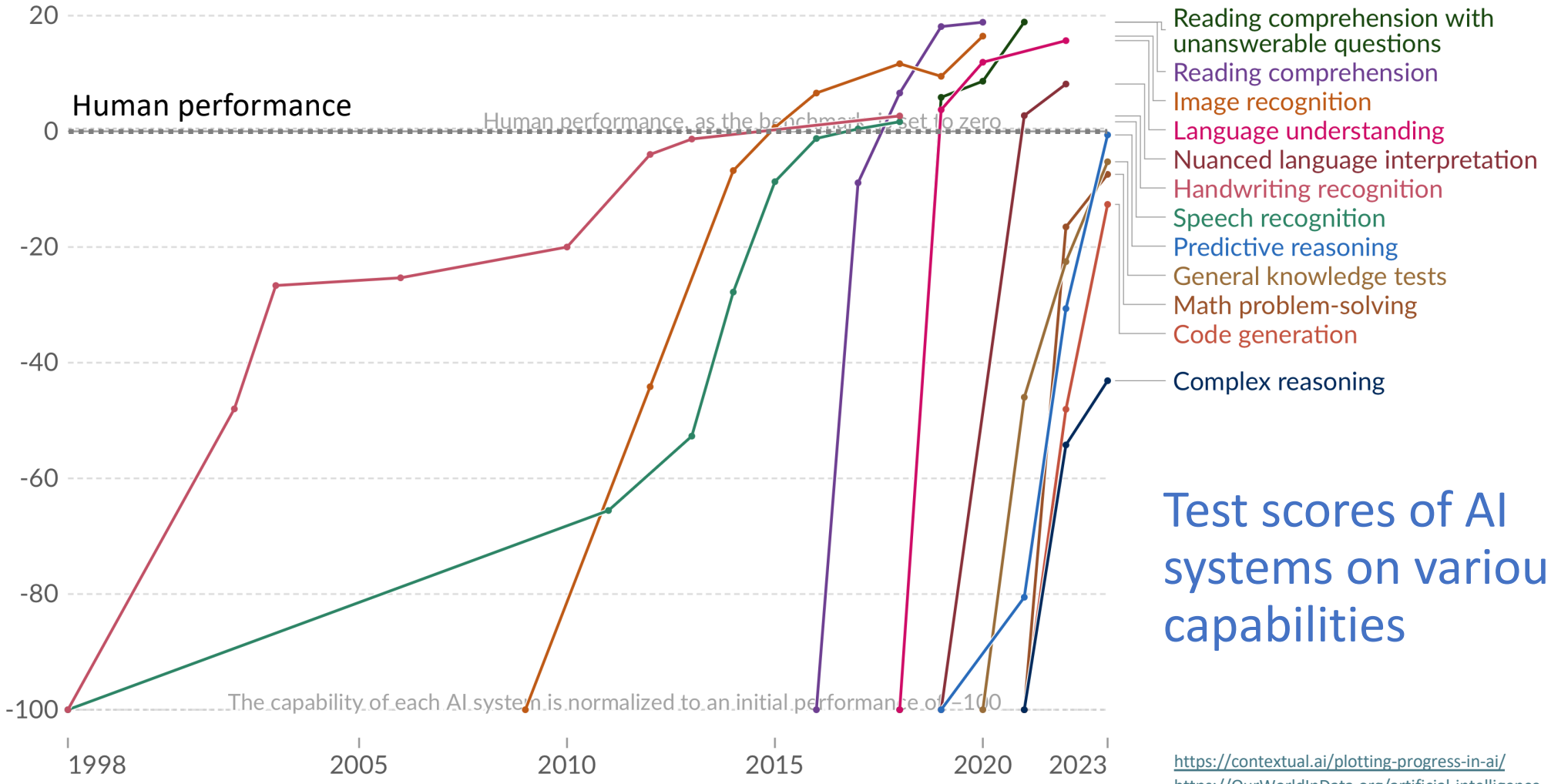
Software Company Multi-Agent Schematic (Gradually Implementing)

U.S. DEPARTMENT OF ENERGY



U.S. DEPARTMENT OF ENERGY

AI system capabilities are increasing rapidly



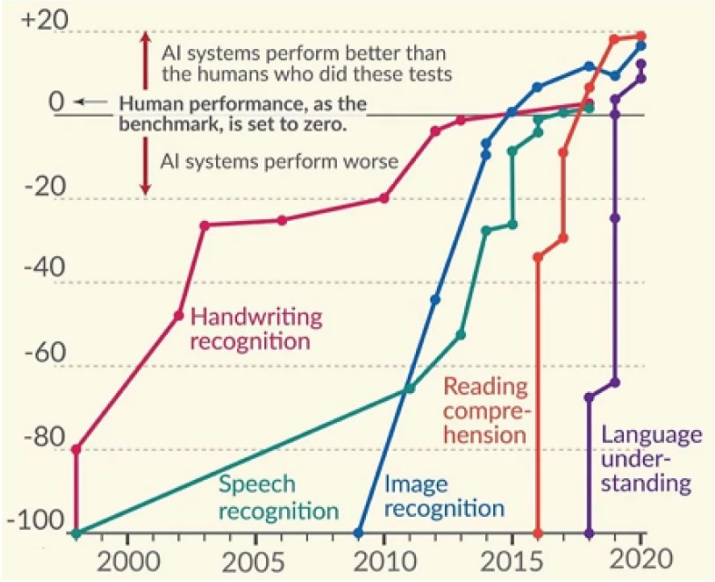
Test scores of AI systems on various capabilities

https://contextual.ai/plotting-progress-in-ai/
https://OurWorldInData.org/artificial-intelligence

An intellect that exceeds that of humans in every important domain

Quality SuperIntelligence

Steady progress in achieving
Superhuman Performance
On increasingly large sets of skills



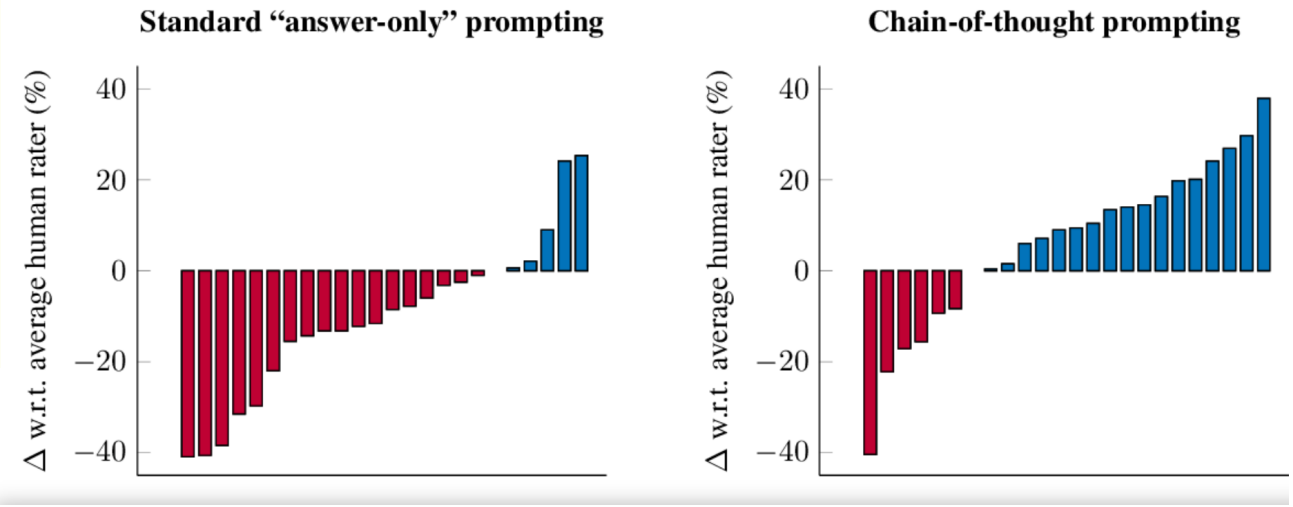


Figure 1: Per-task delta between Codex (code-davinci-002) and the average human-rater performance on 23 challenging tasks in BIG-Bench Hard, for standard "answer-only" (left) and chain-of-thought (right) prompting.

Scaling up LLMs from todays
~ 1.5 trillion to 10 trillion or
100 trillion parameters

U.S. DEPARTMENT OF ENERGY

Big Bench Hard Progress due to use of Chain of Thought prompting

Each of these dimensions will require massive increase in computing capabilities

Speed ⇢ 100x – 1000x ⇢ 3 years in 1 day

Collective ⇢ 10x – 100x ⇢ Team for each person

Quality ⇢ 100x – 1000x ⇢ Everyone a polymath


Mini update on FASST and AI4SES

Data

Scientific & Engineering Datasets

Mathematics
Biology
Materials
Chemistry
Particle Physics
Nuclear Physics
Computer Science
Climate
Medicine
Cosmology
Fusion Energy
Accelerators
Reactors
Energy Systems
Manufacturing

Text and Code Corpora

General Text
Media
News
Humanities
History
Law
Digital Libraries
OSTI Archive
Scientific Journals
arXiv
Code repositories
Data.gov
PubMed
Agency Archives

DOE and NNSA Exascale Systems
FASST Common AI Software Frameworks
FASST Responsible AI Techniques

Compute



Training





Training



Models

Open Science Foundation Models

National Security Foundation Models

Integrated Research Infrastructure
Online Experimental Facilities
Strategic Partnerships

Applications

Tuned and Adapted Downstream Models

Exemplar DOE Mission Tasks

Scientific Discovery

Digital Twins

Inverse Design

Code Optimization

Accelerated Simulations

Autonomous Experiments

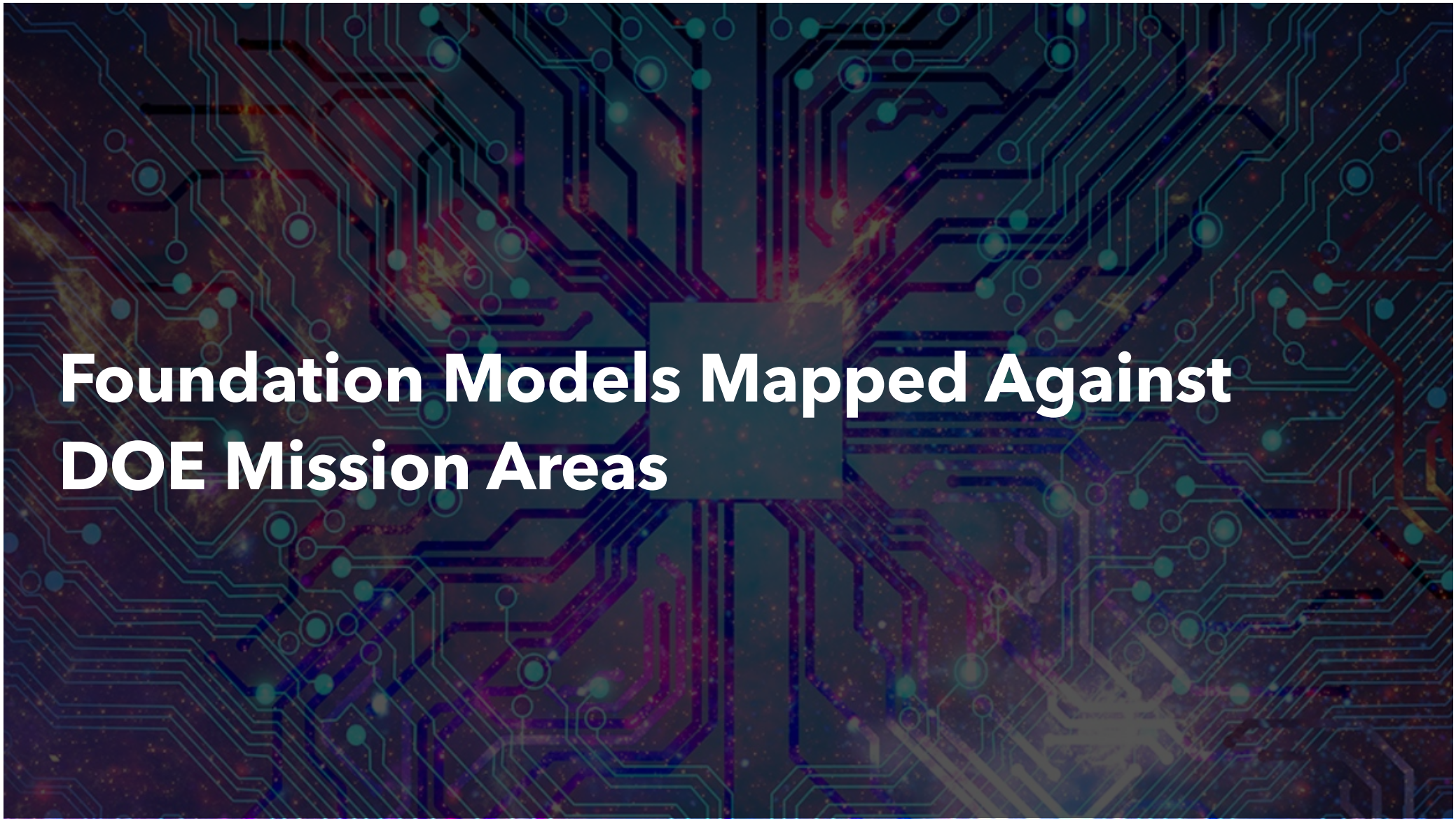
Secure Data Infrastructure

Co-Design

U.S. DEPARTMENT OF ENERGY

Example AI Grand Challenges in Science

- **Purpose driven design of proteins**
  - Room temperature biocatalysis for CO_2 capture and conversion
- **Improved processes in climate models at fraction of the computational costs**
  - Clouds, Biological and Geological processing, atmospheric chemistry
- **Automatically synthesizing and characterizing new quantum materials**
  - Efficient AI driven search for new materials using inverse design
- **Controlling a fusion reactor to sustain long plasma burns**
  - Going beyond classical control regimes
- **Porting scientific codes for new accelerators without human intervention**
  - Improving code development for Science and Security applications
- **Accelerated and automatic interpretation of data from particle detectors**
  - Self-tuning and self-calibrating workflows, accelerated detector simulations
- **Automated high-throughput workflows for light sources and neutron sources**
  - Faster than human in the loop experiments
- **AI driven workflows for data integration, discovery and summarization**
  - Improving working across massive datasets and multiple facilities


Foundation Models Mapped Against
DOE Mission Areas

Clean Energy Systems Model: This model would focus on the physics, chemistry, and engineering principles underlying various renewable energy sources, hydrogen production, solar energy, wind energy, and storage technologies. It can be fine-tuned for specific energy systems, optimization of energy output, and efficiency improvements.

Smart Grid and Infrastructure Model: This model would encompass electrical engineering, network optimization, smart grid technologies, and energy systems management. It can be fine-tuned for specific applications like EV infrastructure planning, grid resilience strategies, and smart grid implementations.

Computational Intelligence Model: This model would integrate capabilities in high-performance computing, machine learning, quantum algorithms, computer science, mathematics, computer architecture, data science, advanced data analysis, applied mathematics, and parallel computing. It can be fine-tuned for applications in AI, complex simulations, and computational research.

Environmental Sciences Model: This model would focus on climate modeling, environmental impact assessments, atmospheric science, climate mitigation strategies, climate risk assessment, bio-geosphere interactions, climate engineering, and biological systems in the environment. It can be fine-tuned for specific environmental applications and climate studies.

Materials and Chemical Sciences Model: This model would cover computational chemistry, materials discovery, molecular dynamics, manufacturing processes, inverse design of materials and systems, self-driving laboratories, and autonomous discovery. It can be fine-tuned for developing new materials, chemical processes, and manufacturing techniques for energy applications.

U.S. DEPARTMENT OF ENERGY

Biological Systems Model: This model would cover the study of biological systems, including genomics, synthetic biology, microbiology, environmental biology, engineering plants, medicine, protein design, self-driving laboratories, autonomous discovery, and microbial engineering. It can be fine-tuned for applications in environmental biology, plant engineering, biotechnology, and medical research.

Nuclear Security Model: This model would integrate nuclear physics, engineering, security protocols, reactor technologies, nuclear fission, automated reactor design, and reactor control. It can be fine-tuned for nuclear energy applications, nonproliferation technologies, and national security measures.

High-Energy and Particle Physics Model: This model would focus on the principles of high-energy physics, nuclear reactions, particle physics, accelerators, and cosmology. It can be fine-tuned for applications in experimental physics, particle accelerators, and fundamental research in physics.

Advanced Manufacturing Model: This model would focus on manufacturing technologies, including inverse design, process optimization, supply chain optimization, applied materials, precision manufacturing, self-driving laboratories, and autonomous discovery. It can be fine-tuned for specific applications in optimizing manufacturing processes and supply chains.

Carbon Management Model: This model would integrate knowledge on the physics and chemistry of capturing CO2, managing CO2 flows, carbon storage, conversion to fuels, and direct air capture. It aims to support the design and analysis of CO2 management systems and advance research into the fundamentals of carbon management.

Knowledge Integration Model: This model would integrate scientific literature, codes, texts, and tutorials to support knowledge extraction, synthesis, and automated hypothesis generation. It aims to advance theory and experimental design, forming the core of a system that interacts with humans and manages interactions with other foundation models included in this list.

Computational Intelligence Model: This model would integrate capabilities in high-performance computing, machine learning, quantum algorithms, computer science, mathematics, computer architecture, data science, advanced data analysis, applied mathematics, and parallel computing. It can be fine-tuned for applications in AI, complex simulations, and computational research.

Knowledge Integration Model: This model would integrate scientific literature, codes, texts, and tutorials to support knowledge extraction, synthesis, and automated hypothesis generation. It aims to advance theory and experimental design, forming the core of a system that interacts with humans and manages interactions with other foundation models included in this list.

Key points on the FASST program

- **Data** – aggregating, cleaning, curating, transforming the many 100's of petabytes of scientific data for AI training/testing, five-year **target of 1000 Trillion tokens**
- **Platforms** – investing to create new platforms for training and inference, investments to push 1000 AI EF @100MW, deployment of **multiple AI frontier training platforms** and **many AI inference optimized systems** – distinct from general HPC
- **Models** – ramping up to train order dozen domain oriented FMs each year to cover science, energy and national security
- **Applications** – downstream adaptation of models for 100's-1000's of DOE use cases


Thank You for Listening

Key Transformer Paper

arXiv:1706.03762v5 [cs.CL] 6 Dec 2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Introduction to JAX

Compiling machine learning programs via high-level tracing

Roy Frostig*
Google Brain
frostig@google.com

Matthew James Johnson*
Google Brain
mattjj@google.com

Chris Leary
Google
leary@google.com

ABSTRACT

We describe JAX, a domain-specific tracing JIT compiler for generating high-performance accelerator code from pure Python and Numpy machine learning programs. JAX uses the XLA compiler infrastructure to generate optimized code for the program subroutines that are most favorable for acceleration, and these optimized subroutines can be called and orchestrated by arbitrary Python. Because the system is fully compatible with Autograd, it allows forward- and reverse-mode automatic differentiation of Python functions to arbitrary order. Because JAX supports structured control flow, it can generate code for sophisticated machine learning algorithms while maintaining high performance. We show that by combining JAX with Autograd and Numpy we get an easily programmable and highly performant ML system that targets CPUs, GPUs, and TPUs, capable of scaling to multi-core Cloud TPUs.

1 INTRODUCTION

Unlocking machine FLOPs has powered the explosion of progress in machine learning. Since the landmark work of AlexNet on dual-GPUs [5], the field has come a long way both in the number of FLOPs available to researchers and the ease with which these FLOPs can be harnessed. The JAX compiler aims to push further in this direction, enabling researchers to write Python programs—leaning on

```python
import autograd.numpy as np
from autograd import grad
from jax import jit_ps

def predict(params, inputs):
  for W, b in params
    outputs = np.dot(inputs, W) + b
    inputs = np.tanh(outputs)
  return outputs

def loss(params, inputs, targets):
  preds = predict(params, inputs)
  return np.sum((preds - targets)**2)

grad_fun = jit_ps(grad(loss))  # Compiled gradient-of-loss function
```

Listing 1: A basic fully-connected neural network with JAX.

XLA compiler infrastructure. Tracing in JAX is high-level both in the sense (i) that it is implemented as user-level code within the source language, rather than as part of the source language's implementation, and (ii) that the trace primitives are not VM-level operations on basic data, but rather library-level numerical functions on array-level data, like matrix multiplies, convolutions, reductions along axes, elementwise operations, and multidimensional indexing and slicing [2, 9, 11].

JAX is built atop the same tracing library used within Autograd, which, being designed for self-closure, recognizes its own operations as primitives. JAX also has Numpy's numerical functions

Review Paper Covering Many Transformers

A Survey of Transformers

TIANYANG LIN, YUXIN WANG, XIANGYANG LIU, and XIPENG QIU*, School of Computer Science, Fudan University, China and Shanghai Key Laboratory of Intelligent Information Processing, Fudan University, China

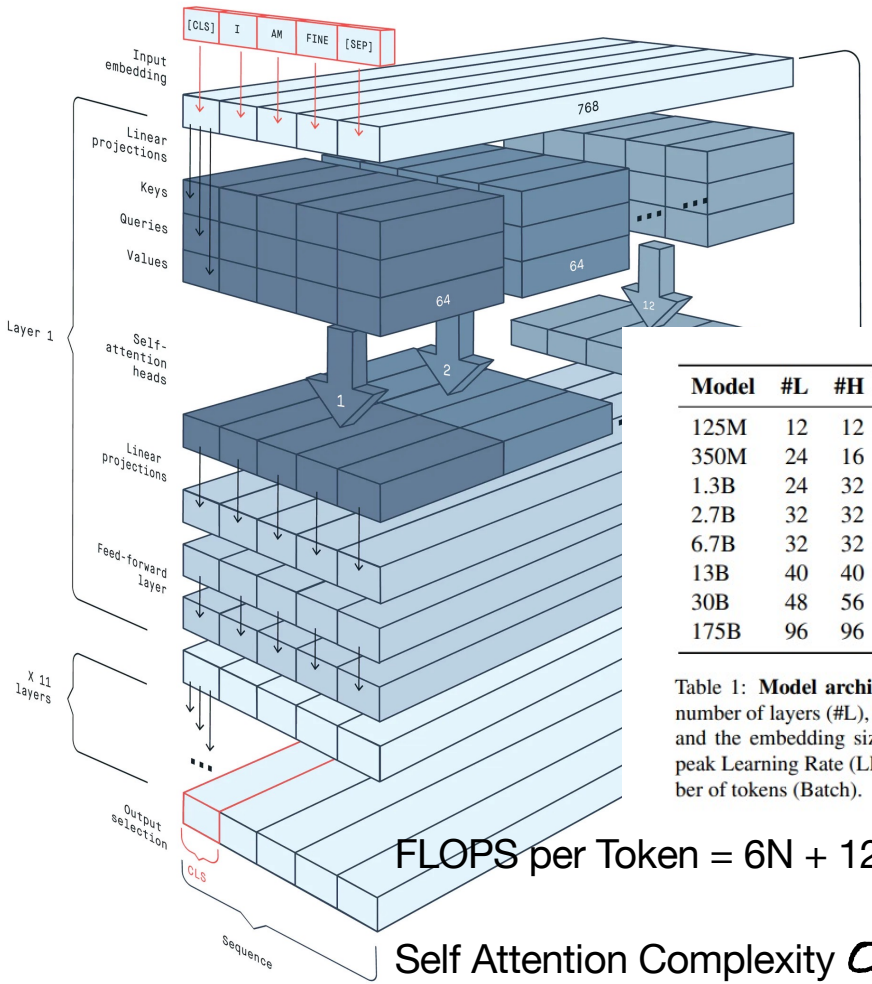
Transformers have achieved great success in many artificial intelligence fields, such as natural language processing, computer vision, and audio processing. Therefore, it is natural to attract lots of interest from academic and industry researchers. Up to the present, a great variety of Transformer variants (a.k.a. X-formers) have been proposed, however, a systematic and comprehensive literature review on these Transformer variants is still missing. In this survey, we provide a comprehensive review of various X-formers. We first briefly introduce the vanilla Transformer and then propose a new taxonomy of X-formers. Next, we introduce the various X-formers from three perspectives: architectural modification, pre-training, and applications. Finally, we outline some potential directions for future research.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Computing methodologies** → **Artificial intelligence**.

Additional Key Words and Phrases: Transformer, Self-Attention, Pre-trained Models, Deep Learning

s.LG] 15 Jun 2021

The Transformer with Self-Attention

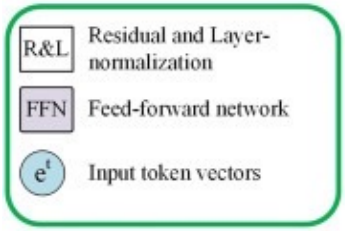


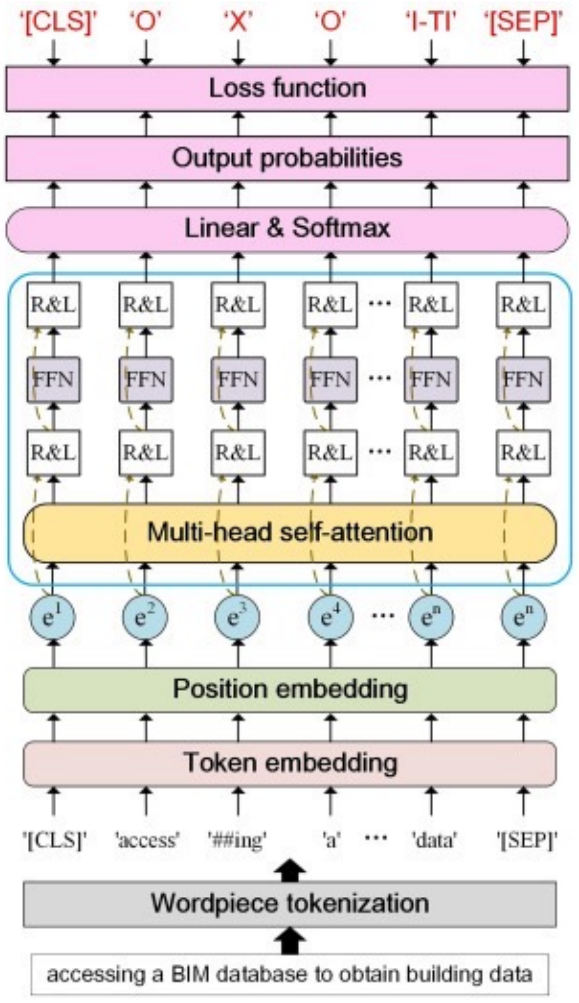
Model	#L	#H	d_{model}	LR	Batch
125M	12	12	768	$6.0e{-4}$	0.5M
350M	24	16	1024	$3.0e{-4}$	0.5M
1.3B	24	32	2048	$2.0e{-4}$	1M
2.7B	32	32	2560	$1.6e{-4}$	1M
6.7B	32	32	4096	$1.2e{-4}$	2M
13B	40	40	5120	$1.0e{-4}$	4M
30B	48	56	7168	$1.0e{-4}$	4M
175B	96	96	12288	$1.2e{-4}$	2M

Table 1: **Model architecture details.** We report the number of layers (#L), number of attention heads (#H), and the embedding size (d_{model}). We also report the peak Learning Rate (LR) and global batch size in number of tokens (Batch).

FLOPS per Token = 6N + 12LHQT

Self Attention Complexity $O(n^2d + nd^2)$





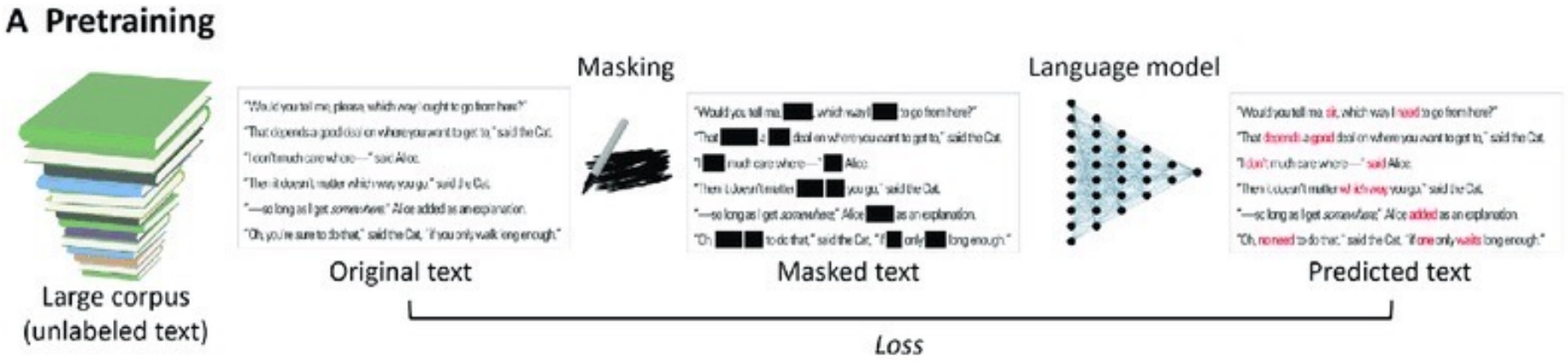
Layers, Heads, Dimension, Width

Computational Complexity of Some Layers

Table 2. Per-layer complexity, minimum number of sequential operations and maximum path lengths for different layer types. T is the sequence length, D is the representation dimension and K is the kernel size of convolutions [137].

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(T^2 \cdot D)$	$O(1)$	$O(1)$
Fully Connected	$O(T^2 \cdot D^2)$	$O(1)$	$O(1)$
Convolutional	$O(K \cdot T \cdot D^2)$	$O(1)$	$O(\log_K(T))$
Recurrent	$O(T \cdot D^2)$	$O(T)$	$O(T)$

Self-Supervised Pretrained Autoregressive Models



0. What is self-attention?

If you're thinking if self-attention is similar to **attention**, then the answer is yes! They fundamentally share the same concept and many common mathematical operations.

A self-attention module takes in n inputs, and returns n outputs. What happens in this module? In layman's terms, the self-attention mechanism allows the inputs to interact with each other ("self") and find out who they should pay more attention to ("attention"). The outputs are aggregates of these interactions and attention scores.